Common Core: Phase 1 RTL Design

Lab R1: Sequential Multiplier

SystemVerilog RTL Constructs and Simulation–Synthesis-Safe Coding

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Design a shift-and-add multiplier with valid/ready handshakes in synthesizable SystemVerilog, following the team coding rules, keep it lint-clean, and show that it works with a small directed self-checking smoke testbench.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Lab L0
Designs	`seq_mul`, `comb_mul`, `mul_pkg` (new)
Reused in	V1 (verification), P1 and P2 (synthesis, timing), T-RTL3

Contents

1 Overview **3**

2 Sequential Multiplier: Terminology and Architecture **3**

2.1 Right-shift and left-shift architectures . . . 3

2.2 From the diagram to your RTL . . . 5

3 Design Specification **6**

3.1 Operations . . . 7

3.2 Package `mul_pkg` . . . 7

3.3 Interface . . . 8

3.4 Behavior . . . 8

3.5 Combinational Reference Module . . . 8

4 Required SystemVerilog Constructs **9**

5 RTL Coding Guidelines: Simulation–Synthesis Consistency **9**

5.1 Required Coding Style: lowRISC Verilog/SystemVerilog Guide . . . 9

5.2 Rule 1: Assign every output on every path of a combinational block . . . 10

5.3 Rule 2: Use `always_comb`, never a hand-written sensitivity list . . . 10

5.4 Rule 3: Nonblocking (`<=`) in `always_ff`, blocking (`=`) in `always_comb` . . . 10

5.5 Rule 4: One variable, one always block . . . 11
5.6 Rule 5: In `always_comb`, write a variable before you read it . . . 11
5.7 Rule 6: Reset with a reset branch, never with initializers . . . 11
5.8 Rule 7: Do not let X hide bugs . . . 12
5.9 Rule 8: Use `unique` / `priority`, never `full_case` / `parallel_case` pragmas . . . 12
5.10 Rule 9: Constructs that synthesis ignores . . . 13
5.11 Rule 10: Loops and functions describe hardware, not time . . . 13
5.12 Rule 11: FSM coding style . . . 13
5.13 Rule 12: Width and sign (simulation and synthesis agree, but both are wrong) . . . 13
5.14 Lint Checklist . . . 14

6 Exercise: Seeing the Rules in Lint **14**

7 Smoke Testbench **16**
7.1 Required Structure . . . 16
7.2 Avoiding Testbench–DUT Races . . . 16
7.3 Required Tests . . . 17
7.4 Skeleton . . . 17

8 Requirements and Tasks **19**

9 Repository and Flow **20**
9.1 Files created in this lab . . . 20
9.2 File lists . . . 20
9.3 IP Makefile . . . 21
9.4 Commands . . . 21

10 Deliverables **22**

11 Assessment **22**

12 Review Questions **22**

13 References **23**

1 Overview

Every block on a chip starts as RTL written by a designer, and that RTL is read by two very different programs: a simulator, which executes it like software, and a synthesis tool, which turns it into gates and flip-flops. A design team can only trust its simulations if the RTL means the same thing to both. This lab teaches the coding discipline that guarantees this, on a design small enough to understand completely.

You will design a sequential (shift-and-add) multiplier, `seq_mul`, that implements the RISC-V M-extension multiply operations with valid/ready handshakes on its input and output, and a purely combinational reference multiplier, `comb_mul`. You will keep both lint-clean and prove that `seq_mul` works with a small directed self-checking testbench — the kind of smoke test every designer writes before handing a block to verification.

Where this lab sits. Lab R1 builds on the repository, file lists and `make` flow from Lab L0: the multiplier is the first real IP in `hw/ip/seq_mul/`. Your RTL is reused unchanged by later labs: Lab V1 verifies it with a full self-checking environment, random stimulus and assertions; Labs P1 and P2 synthesize and time it (and compare it with `comb_mul`); Lab T-RTL3 pipelines it to meet a frequency target. Write it as if other people will depend on it — they will.

By the end of this lab you should be able to:

1. Explain the shift-and-add algorithm and map its registers and adder onto an RTL datapath controlled by an FSM.
2. Write synthesizable RTL using SystemVerilog constructs (packages, enums, packed structs, parameters, `always_ff`, `always_comb`, `unique case`, `function automatic`).
3. Apply the lowRISC coding style and the twelve coding rules of this lab, and explain which coding styles cause simulation–synthesis mismatches and why.
4. Use Verilator lint as a design tool: read its warnings, map them to the coding rules and fix or justify every one of them.
5. Implement a valid/ready handshake whose latency the environment does not need to know.
6. Write a directed, self-checking smoke testbench with a reference function, reset and stimulus tasks, a report and a watchdog.

2 Sequential Multiplier: Terminology and Architecture

Before implementing the RTL, identify the operands and the main hardware blocks. For the multiplication $P = X \times Y$:

- **Multiplicand (M):** the value added to the partial result when the current multiplier bit is 1.
- **Multiplier (Q):** the operand whose least-significant bit (Q_0) is examined on each iteration.
- **Accumulator (A):** stores the intermediate sum.
- **Carry (C):** stores the carry-out from the addition and participates in the combined right shift.
- **Product (P):** the final multiplication result. In the classic shift-and-add architecture shown below, it is read from the combined accumulator and multiplier registers after the required iterations.

2.1 Right-shift and left-shift architectures

Both diagrams show a serial multiply: one reusable adder performs one conditional add per clock, while a counter tracks the iterations. They place the shifts in different registers. The right-shift version

moves the combined carry, accumulator and multiplier; the left-shift version shifts the multiplicand left and the multiplier right. These are alternative datapath organizations, not two steps to combine.

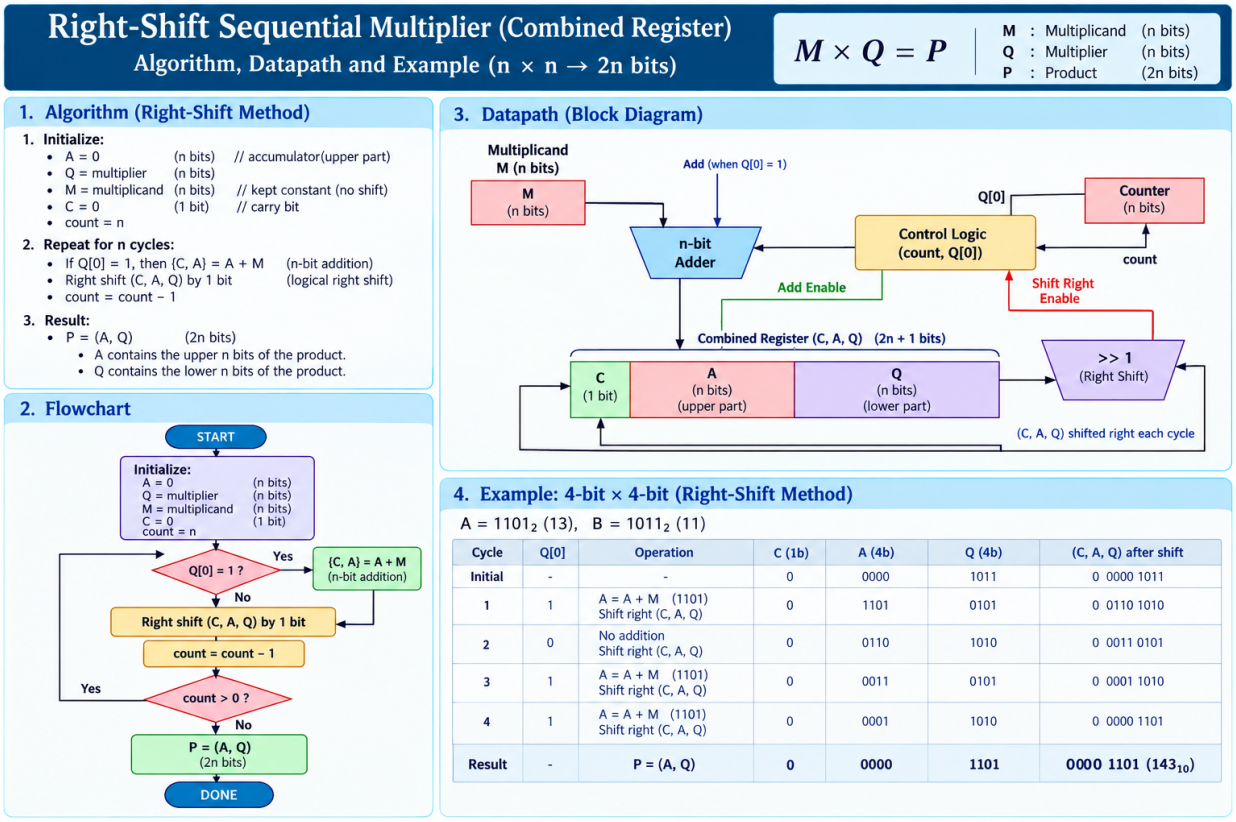


Figure 1: Right-shift sequential multiplier: the combined (C, A, Q) register shifts right after each conditional addition.

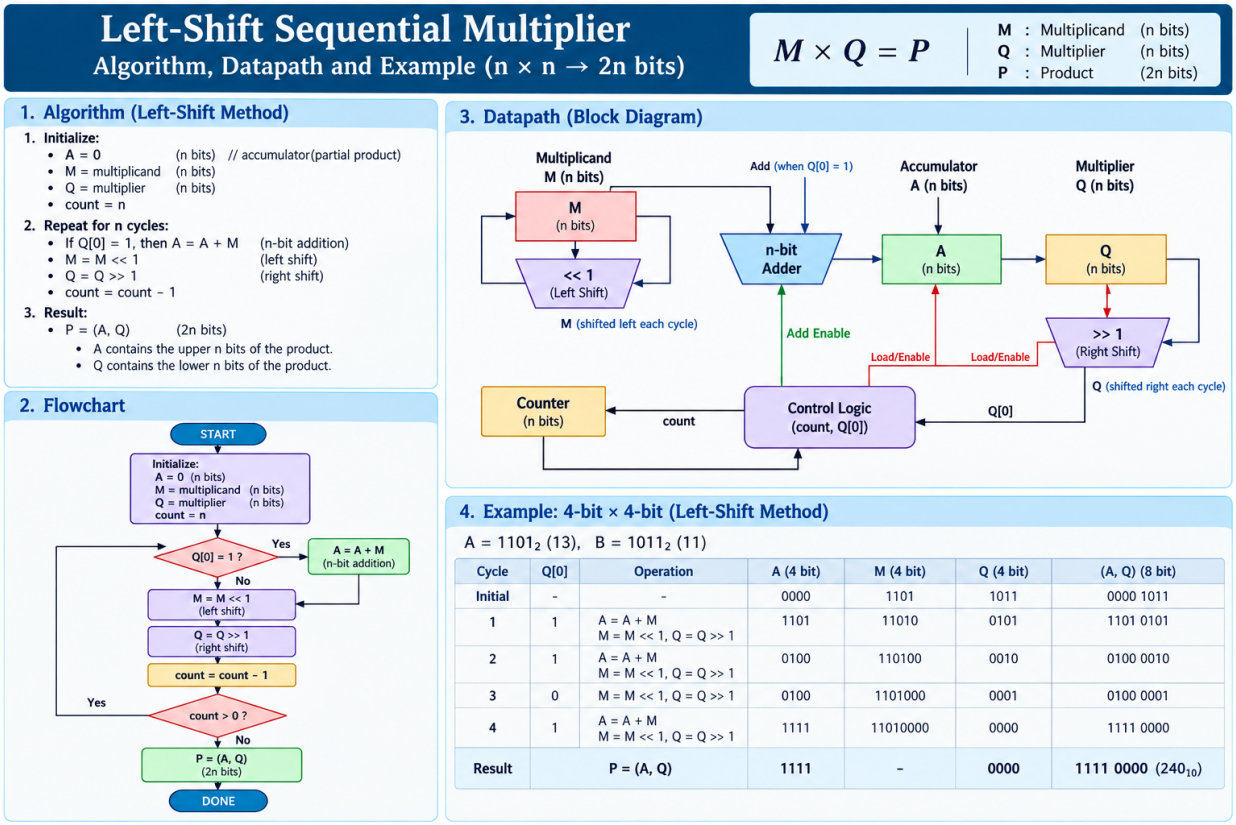


Figure 2: Left-shift organization: the multiplicand shifts left while the multiplier shifts right. A practical implementation must retain enough width for the shifted multiplicand and partial sum.

This lab implements the right-shift organization in Figure 1. The two supplied diagrams are useful for comparing register movement, but their worked-example tables do not give a reliable arithmetic trace. Use the following corrected trace for 13×11 (four bits):

Iteration	Q_0	Add result (C, A)	After shift C	After shift A	After shift Q
Initial	–	–	0	0000	1011
1	1	0, 1101	0	0110	1101
2	1	1, 0011	0	1001	1110
3	0	unchanged	0	0100	1111
4	1	1, 0001	0	1000	1111

The final pair $(A, Q) = 1000\,1111_2 = 143_{10}$. The carry-out from an addition is shifted into the accumulator in that same iteration; it is not discarded. The actual `seq_mul` also has a ready/valid interface, an FSM, and RISC-V signed/unsigned operation selection, so the figures are algorithm references rather than pin-level schematics.

2.2 From the diagram to your RTL

A good way to start is to write down, before any code, every register of your design and why it exists. A typical mapping is:

Concept (Figure 1)	**RTL element**	**Notes**
Multiplicand register M	`mcand_q` (XLEN bits)	Loaded when a request is accepted; datapath register, may skip reset (Rule 6)
(C, A, Q) shift register	`prod_q` (2·XLEN+1 bits)	Loaded with $\{0, 0, b\}$; shifted right once per iteration
n-bit adder	One `+` in `always_comb`	The only arithmetic in the datapath; reused every cycle
Sequence counter	`cnt_q`, `$clog2(XLEN)` bits	Counts the XLEN iterations
Control logic	FSM `state_q`: `S_IDLE`, `S_BUSY`, `S_DONE`	Generates `in_ready`, `out_valid`, load and shift enables
Operation	`op_q`	Selects the lower or upper half at the output

Figure 3 shows one implementation of this mapping as a block diagram, the level of detail expected in your own `doc/design.md`.

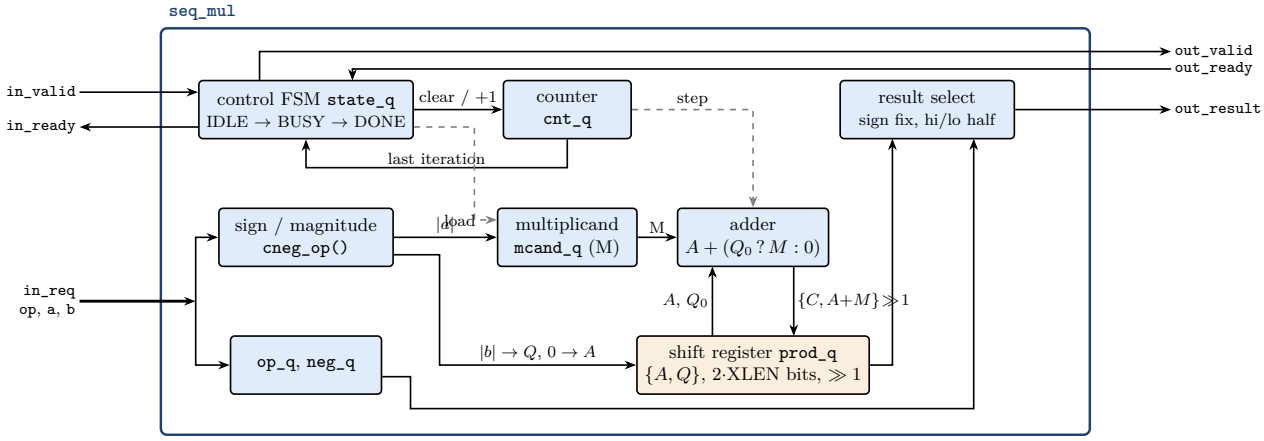


Figure 3: Block diagram of a `seq_mul` implementation (blue: logic, orange: the shift register that holds the partial product). On acceptance the FSM loads $|a|$ into M, $\{0, |b|\}$ into `prod_q` and the sign of the product into `neg_q`; in `S_BUSY` one add-and-shift step per cycle for XLEN cycles; in `S_DONE` `out_result` is the sign-corrected lower or upper half, held until `out_ready`. The carry C is shifted into A in the same cycle, so it needs no flip-flop.

Your `doc/design.md` must contain your own version of this table, a datapath block diagram, the FSM state diagram, and the exact latency in cycles from request acceptance to `out_valid`.

3 Design Specification

The main module is `seq_mul`: a multiplier that computes one partial product per clock cycle using the shift-and-add algorithm, with valid/ready handshakes on its input and output. Copy this section into `hw/ip/seq_mul/doc/spec.md` and keep the two consistent.

3.1 Operations

Operation	Result	Operand types	Status
MUL	Lower XLEN bits of $a \times b$	Signed or unsigned (lower half is identical)	Required
MULHU	Upper XLEN bits of $a \times b$	Unsigned $\times$ unsigned	Required
MULH	Upper XLEN bits of $a \times b$	Signed $\times$ signed	Stretch goal
MULHSU	Upper XLEN bits of $a \times b$	Signed a $\times$ unsigned b	Stretch goal

For the stretch goals, one approach is to multiply the absolute values and negate the 2×XLEN-bit product when the result must be negative. Check the exact definitions in the RISC-V unprivileged specification (M extension) before you start.

3.2 Package `mul_pkg`

Shared types and constants live in one package that both the RTL and the testbench import. The operation encoding follows the low two bits of the RISC-V `funct3` field, so the unit can be connected to a decoder without translation.

```
// hw/ip/seq_mul/rtl/mul_pkg.sv
`ifndef MUL_XLEN
`define MUL_XLEN 32
`endif

package mul_pkg;
  localparam int unsigned XLEN = `MUL_XLEN;

  typedef enum logic [1:0] {
    OP_MUL    = 2'b00,
    OP_MULH   = 2'b01,
    OP_MULHSU = 2'b10,
    OP_MULHU  = 2'b11
  } mul_op_e;

  typedef struct packed {
    mul_op_e         op;
    logic [XLEN-1:0] a;
    logic [XLEN-1:0] b;
  } mul_req_t;
endpackage
```

3.3 Interface

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low
`in_valid`	input	1	Request is valid
`in_ready`	output	1	Unit can accept a request
`in_req`	input	`mul_req_t`	Packed struct: `op`, `a`, `b`
`out_valid`	output	1	Result is valid
`out_ready`	input	1	Consumer accepts the result
`out_result`	output	XLEN	Selected half of the product

Width: `XLEN` is a `localparam` in `mul_pkg`, set from the macro `MUL_XLEN` (default 32), because the packed struct `mul_req_t` needs it. Your design must also work when compiled with `MUL_XLEN=8` (`make ... DEFINES=MUL_XLEN=8`), which makes exhaustive testing possible.

3.4 Behavior

1. After reset the unit is idle: `in_ready = 1`, `out_valid = 0`.
2. A request is accepted on a rising edge where `in_valid && in_ready`. The unit then deasserts `in_ready`.
3. The unit performs XLEN iterations, one per cycle.
4. It then asserts `out_valid` and holds `out_result` stable until a rising edge where `out_valid && out_ready`.
5. After the result is accepted, the unit returns to idle.

The environment must not assume a fixed latency. It must rely only on the handshake signals. This is what allows Lab T-RTL3 to replace the unit by a pipelined one without changing the testbench of Lab V1.

3.5 Combinational Reference Module

Also write `comb_mul`: registered inputs, a single `a * b` expression, registered output. It is not used in any system; it exists so that Lab P2 can compare area, timing and throughput of the two implementations of the same function. It does not need a handshake interface.

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low
`a`, `b`	input	XLEN	Unsigned operands, registered inside
`p`	output	2·XLEN	Full unsigned product, registered (two cycles after `a`, `b`)

`comb_mul` imports `mul_pkg` for `XLEN` and lives in the same file list as `seq_mul`. It is linted with `make lint DUT_TOP=comb_mul` and gets a short directed check in the smoke testbench (Section 7).

4 Required SystemVerilog Constructs

Every construct below must appear in your RTL, and you must be able to explain why it was the right choice at that point.

Construct	Where to use it	Why
`package` (`mul_pkg`)	Shared types and constants	One definition used by RTL and testbench
`typedef enum logic [..]`	Operation code and FSM state	Readable names, waveform shows state names, explicit encoding width
`typedef struct packed`	`mul_req_t` request bundle	Groups related fields; still a plain bit vector for synthesis
`parameter` / `localparam`	`XLEN`, counter width via `$clog2(XLEN)`	One design for every width
`logic`	All signals	Replaces `reg`/`wire` confusion
`always_ff`	All flip-flops	Tool checks that the block infers only flops
`always_comb`	All combinational logic	Correct sensitivity, executes at time 0, tool warns on latches
`unique case`	FSM next-state and output decode	States the intent that exactly one item matches; simulator reports violations
`function automatic`	One small helper in the RTL (for example, conditional two's-complement negation)	Reusable combinational logic without static-variable surprises

Not allowed in RTL: `initial` blocks, `#` delays, variable initializers in declarations (`logic x = 0;`), `casex`, `full_case` / `parallel_case` pragmas, and `translate_off` / `translate_on` pragmas. The next section explains why.

5 RTL Coding Guidelines: Simulation–Synthesis Consistency

5.1 Required Coding Style: lowRISC Verilog/SystemVerilog Guide

Use the lowRISC Verilog/SystemVerilog Coding Style Guide as the default coding standard for all RTL in this program, starting with this lab: lowRISC Verilog Coding Style Guide. It is the style used by OpenTitan and Ibex, so the open-source code you will read later looks like yours.

Follow the guide for naming (for example `_q` / `_d` suffixes for register outputs and next values, `_n` for active-low signals, `_e` / `_t` for enum and struct types), formatting, sequential and combinational processes, case statements, reset logic, and synthesizable RTL. If a lab requirement or tool limitation conflicts with the guide, document the specific deviation in the merge request or README rather than silently introducing a different style. The guide is a coding standard; it does not replace the functional specification, lint checks, or the simulation–synthesis consistency rules in this section.

A simulator executes your code line by line as a program; a synthesis tool reads the same code as a description of gates and flip-flops. When the two interpretations differ, RTL simulation passes but the silicon behaves differently. Each rule below closes one of those gaps. Follow all of them in this lab and be ready to explain each one in the review. You will *see* these mismatches happen on real netlists in Lab P1; in this lab you prevent them by construction and catch what you can with lint.

5.2 Rule 1: Assign every output on every path of a combinational block

If some path through an `always_comb` block does not assign a variable, the variable must keep its old value — that is memory, so synthesis infers a latch. Simulation and synthesis agree here, but the latch is unintended hardware that breaks timing analysis and testability. There are three ways to prevent it.

```
// BAD: grant is not assigned when req == 0 -> latch
always_comb begin
  if (req) grant = 1'b1;
end

// GOOD, style 1 (recommended): default value at the top of the block
always_comb begin
  grant = 1'b0;
  if (req) grant = 1'b1;
end

// GOOD, style 2: an else on every if
always_comb begin
  if (req) grant = 1'b1;
  else     grant = 1'b0;
end

// GOOD, style 3: a default item in every case
always_comb begin
  unique case (op)
    OP_MUL:   sel_hi = 1'b0;
    OP_MULHU: sel_hi = 1'b1;
    default:  sel_hi = 1'b0;
  endcase
end
```

Style 1 scales best. In a block with ten outputs and a nested case, styles 2 and 3 require every branch to assign all ten outputs; one forgotten assignment creates a latch. With defaults at the top, each branch only assigns what it changes. Use style 1 for FSM next-state logic: `state_d = state_q;` as the first line.

5.3 Rule 2: Use `always_comb`, never a hand-written sensitivity list

```
// BAD (Verilog style): b is missing from the sensitivity list
always @(a) y = a & b;
// Simulation: y does not update when only b changes.
// Synthesis: ignores the list and builds an AND gate.  -> mismatch

// GOOD
always_comb y = a & b;
```

`always_comb` is also better than `always @*`: it executes once at time zero, it is sensitive to signals read inside called functions, and the tool reports an error if another block writes the same variable.

5.4 Rule 3: Nonblocking (`<=`) in `always_ff`, blocking (`=`) in `always_comb`

```
// BAD: blocking assignments in sequential blocks
always_ff @(posedge clk) q1 = d;
always_ff @(posedge clk) q2 = q1;
```

```
// Simulation: if the first block runs first, q2 receives the new d
// (one stage); otherwise two stages. The result depends on block order.
// Synthesis: always builds a two-stage shift register.  -> mismatch

// GOOD
always_ff @(posedge clk) begin
  q1 <= d;
  q2 <= q1;
end
```

Nonblocking assignments read all right-hand sides first and update at the end of the time step, exactly like flip-flops sampling on the same edge. Never mix `=` and `<=` in one block. Lab R2 explains precisely why this works, in terms of the simulator's scheduling regions.

5.5 Rule 4: One variable, one always block

```
// BAD: count is written from two blocks
always_ff @(posedge clk) if (clr) count <= '0;
always_ff @(posedge clk) if (inc) count <= count + 1'b1;
// Simulation: the last block to execute wins (order dependent).
// Synthesis: multiple drivers, error or undefined hardware.

// GOOD: one block, explicit priority
always_ff @(posedge clk) begin
  if      (clr) count <= '0;
  else if (inc) count <= count + 1'b1;
end
```

`always_ff` and `always_comb` make this a compile error, which is one more reason to use them.

5.6 Rule 5: In `always_comb`, write a variable before you read it

```
// BAD: tmp is read before it is assigned
always_comb begin
  y   = tmp | c;
  tmp = a & b;
end
// Simulation: always_comb is not sensitive to variables it writes,
// so y uses the stale tmp from the previous evaluation.
// Synthesis: builds y = (a & b) | c.  -> mismatch

// GOOD: statement order follows data flow
always_comb begin
  tmp = a & b;
  y   = tmp | c;
end
```

5.7 Rule 6: Reset with a reset branch, never with initializers

```
// BAD: ASIC synthesis ignores both of these
logic [4:0] count = '0;
initial state = S_IDLE;
// Simulation starts at known values; silicon powers up at random values.
```

```
// GOOD: asynchronous active-low reset
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    state_q <= S_IDLE;
    cnt_q   <= '0;
  end else begin
    state_q <= state_d;
    cnt_q   <= cnt_d;
  end
end
```

Rules for reset:

- The reset edge must be in the sensitivity list, and the reset test must be the first `if`.
- Every control register (state, valid flags, counters) must be reset.
- Pure datapath registers (operands, partial product) may skip reset, but then put them in a separate `always_ff` block. Mixing reset and non-reset registers in one block makes synthesis add an enable on the non-reset ones.

5.8 Rule 7: Do not let X hide bugs

A simulator uses four values (0, 1, X, Z); hardware has only 0 and 1. Simulation can be optimistic about X:

```
always_comb begin
  if (sel) y = a;
  else     y = b;
end
// If sel is X, simulation takes the else branch: y = b, no X visible.
// In hardware sel is 0 or 1, so y may be a.  -> a real bug stays hidden
```

- Reset every control signal so it is never X after reset.
- Verilator, the simulator used in this program, is two-state: X becomes 0 or 1, so an X-related bug can stay invisible. By default an uninitialized flip-flop starts at 0, which can hide a missing reset. Random initialization (`--x-assign unique --x-initial unique` at compile time, `+verilator+rand+reset+2` and a seed at run time) gives every uninitialized variable a random value, as silicon does at power-up. It becomes part of the multi-seed regression in Lab V1 and of gate-level simulation in Lab P1; until then, the best defense is to reset every control register.
- Never use `casex`: an X or Z in the selector is treated as don't-care and matches the wrong item. Use `case inside` when you need wildcard items.
- Never use `===` or `!==` in RTL. Synthesis treats them as `==` and `!=`, so they behave differently once X is involved. They are fine in the testbench.

5.9 Rule 8: Use `unique` / `priority`, never `full_case` / `parallel_case` pragmas

```
// BAD: a comment the simulator ignores but synthesis obeys
case (state_q) // synopsys full_case parallel_case

// GOOD: both tools see the same intent
unique case (state_q)
```

Pragmas let synthesis optimize the hardware under assumptions that simulation never checks. `unique case` states the same assumption in the language, and the simulator reports a violation at runtime when zero or more than one item matches. Synthesis may still optimize unlisted values away, so keep the default assignments from rule 1 and treat every `unique` violation message as a bug. Use `priority if` / `priority case` where the first match must win.

5.10 Rule 9: Constructs that synthesis ignores

- `#` delays: `assign #2 y = a;` simulates with a delay; synthesis produces a plain wire.
- `initial` blocks and declaration initializers (rule 6).
- `// synopsys translate_off` / `translate_on`: the code between them exists only in simulation.
- `$display` and other system tasks: harmless in RTL, but they are not hardware.

If your design only works because of one of these, it does not work.

5.11 Rule 10: Loops and functions describe hardware, not time

```
// This is NOT a sequential multiplier.
// The loop is unrolled into XLEN adders in parallel:
// one very large combinational block that finishes in zero cycles.
always_comb begin
  prod = '0;
  for (int i = 0; i < XLEN; i++)
    if (b[i]) prod = prod + ((2*XLEN)'(a) << i);
end
```

- A `for` loop with constant bounds becomes parallel copies of its body. Iterations over time require a counter and a register.
- Loop bounds must be constants (parameters are fine). A `while` loop with a data-dependent bound is not synthesizable.
- Declare functions `function automatic`. A static function shares its local variables between all calls, which can corrupt values in simulation but not in synthesized logic.

5.12 Rule 11: FSM coding style

- State register in `always_ff`; next-state and outputs in `always_comb`.
- State type: `typedef enum logic [1:0] {S_IDLE, S_BUSY, S_DONE} state_e;` with an explicit width.
- First line of the next-state block: `state_d = state_q;`.
- A `default` item that returns to `S_IDLE`, so an illegal encoding cannot lock the FSM.

5.13 Rule 12: Width and sign (simulation and synthesis agree, but both are wrong)

These do not cause mismatches, but they are the most common source of datapath bugs in a multiplier.

```
logic        [31:0] a, b;
logic signed [31:0] sa;
```

```
logic        [63:0] p;

assign p = a * b;   // OK: the 64-bit context widens the operation
assign p = sa * b;  // BAD: b is unsigned, so the whole expression is unsigned
```

- One unsigned operand makes the whole expression unsigned. Use `$signed()` and explicit sign extension.
- A narrow result silently truncates: `logic [31:0] p32 = a * b;` keeps only the lower half.
- Fix or justify every width warning from lint.

5.14 Lint Checklist

Run `make lint` (Verilator `--lint-only -Wall` on the RTL file list, Lab L0) before every commit. These warnings map directly to the rules above:

Verilator warning	Rule	Meaning
`LATCH`	1	`always_comb` does not assign a variable on every path
`BLKSEQ`	3	Blocking assignment in a sequential block
`COMBDLY`	3	Nonblocking assignment in a combinational block
`MULTIDRIVEN`	4	Variable written from more than one block
`CASEINCOMPLETE`	1, 11	`case` without `default` that misses values
`CASEX`	7	`casex` used
`WIDTH...`	12	Implicit truncation or extension (`WIDTHTRUNC`, `WIDTHEXPAND`)

The RTL must be lint-clean for both `DUT_TOP=seq_mul` and `DUT_TOP=comb_mul`, at the default width and with `DEFINES=MUL_XLEN=8`. Any waiver goes into `hw/ip/seq_mul/lint/seq_mul.vlt`, one line per waiver, each with a comment that justifies it:

```
// hw/ip/seq_mul/lint/seq_mul.vlt
`verilator_config
// prod_q[0] is shifted out and never read; keeping the register a
// uniform (C,A,Q) shift register matches doc/design.md.
lint_off -rule UNUSEDSIGNAL -file "*/seq_mul.sv" -match "*prod_q*"
```

A waiver is a design decision that a reviewer must be able to agree with. "The warning was annoying" is not a justification.

SYNOPSYS CLOUD

Lint with SpyGlass: `make lint LINT_TOOL=spyglass` (the cloud default) runs the `lint/lint_rtl` goal in `sg_shell`; `LINT_TOOL=vcst` runs VC SpyGlass in `vc_static_shell`; waivers go into `lint/seq_mul.awl` (SpyGlass) or `lint/seq_mul.vcst.tcl`. The target fails on SpyGlass errors (`LINT_FAIL_ON=warning` makes warnings fatal too). The RTL must be clean with both tools. The rule sets differ: SpyGlass names rules such as `InferLatch`, `W336` (blocking assignment in a flip-flop block) and `W415` (multiple drivers), and checks much more than Verilator.

6 Exercise: Seeing the Rules in Lint

Reading the rules is not enough; you must see the tool catch a violation. Write a small, deliberately bad module `hw/ip/seq_mul/lint/lint_demo_bad.sv` that is *not* part of any file list. It must contain

one violation of each of the rules 1, 3, 4, 7 and 12, and nothing else that triggers a warning. A starting point:

```
// hw/ip/seq_mul/lint/lint_demo_bad.sv -- deliberately bad, never synthesized
module lint_demo_bad (
  input  logic       clk,
  input  logic       req, clr, inc,
  input  logic [1:0] sel,
  input  logic [7:0] a, b,
  output logic       grant,
  output logic [7:0] q2, count, y,
  output logic [7:0] prod
);
  logic [7:0] q1;

  always_comb if (req) grant = 1'b1;        // Rule 1: latch

  always_ff @(posedge clk) q1 = a;          // Rule 3: blocking in always_ff
  always_ff @(posedge clk) q2 = q1;

  always_ff @(posedge clk) if (clr) count <= '0;            // Rule 4:
  always_ff @(posedge clk) if (inc) count <= count + 1'b1;  // two drivers

  always_comb begin                         // Rule 7: casex
    casex (sel)
      2'b1?:   y = a;
      default: y = b;
    endcase
  end

  assign prod = a * b;                      // Rule 12: 16-bit result truncated
endmodule
```

Run Verilator on it directly (it is not in the IP file list, so `make lint` does not see it):

```
cd $REPO_ROOT/hw/ip/seq_mul
verilator --lint-only -Wall lint/lint_demo_bad.sv 2>&1 | tee \
  $REPO_ROOT/build/seq_mul/lint_demo.log
```

SYNOPSYS CLOUD

Run the same file through SpyGlass and compare: `make lint LINT_FLIST=<a .f with lint_demo_bad.sv> LINT_TOP=lint_demo_bad LINT_TOOL=spyglass`. The two tools do not find the same violations (for example, Verilator 5.013 does not report the two same-clock drivers of rule 4, SpyGlass `lint_rtl` does); record both in the table.

For each of the five rules, record in `doc/design.md` (section "Lint demo"):

1. The exact warning or error text, and the line it points to.
2. Which rule it maps to, and what hardware or simulation behavior the violation would cause.
3. The fix, and proof (re-run) that the warning disappears.

Some violations are reported as errors rather than warnings, and the exact warning names depend on the Verilator version (`WIDTH` was split into `WIDTHTRUNC` and `WIDTHEXPAND` in Verilator 5). If a violation is *not* reported, that is a valid result: explain why lint cannot see it, and which later check would (Lab V1 assertions or Lab P1 gate-level simulation). Note also which of the other rules (2, 5, 6, 8, 9, 10, 11) lint cannot check at all; those are the ones the Lab P1 mismatch experiments make visible on a synthesized netlist.

7 Smoke Testbench

A designer does not hand a block to verification until it does the basic job. The smoke testbench `tb/tb_seq_mul.sv` is that proof: small, directed, self-checking and quick to run after every change. It is *not* the verification of the block — that is Lab V1, which builds a full environment with monitors, a scoreboard, back-pressure, random stimulus and assertions on top of this design.

7.1 Required Structure

Component	Implemented as	Responsibility
Reference model	`function automatic logic [XLEN-1:0] ref_mul(...)`	Compute the correct result in a simple, obviously correct way
Reset	`task automatic apply_reset()`	Set all inputs to idle values, assert `rst_n` for a few cycles, release it
Stimulus and check	`task automatic do_mul(op, a, b)`	Drive one request at the falling edge, wait for the input handshake, wait for the output handshake, compare with `ref_mul`
Report	`function void report()`	Print the number of checks and errors; print exactly one line `TEST PASSED` or `TEST FAILED`
Watchdog	An `initial` block	End the simulation with `TEST FAILED` if it runs too long (a hang is a failure)

Figure 4 shows how the pieces fit together.

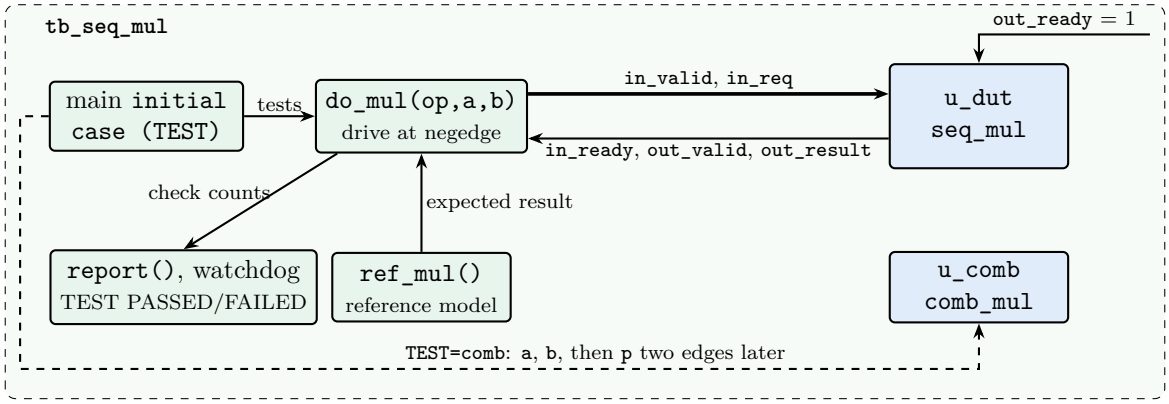


Figure 4: Structure of the R1 smoke testbench: one stimulus-and-check task drives a request at the falling edge, waits for both handshakes and compares with the reference function; `out_ready` is held at 1 (back-pressure is a Lab V1 topic).

The testbench prints its configuration (`TEST`, `XLEN`) at the start, as every simulation in this program does. `out_ready` is held at 1; back-pressure is tested in Lab V1.

7.2 Avoiding Testbench–DUT Races

If the testbench changes an input on the same clock edge on which the DUT samples it, the result depends on simulator scheduling. In this lab, drive DUT inputs at the falling edge and read DUT outputs at the rising edge; then the DUT never samples a value that changes at the same instant. Lab R2 explains the scheduling rules behind this, and introduces the standard used from then on (driving with `<=` at the rising edge).

7.3 Required Tests

The test is selected with `+TEST=` (the `TEST` variable of the flow). An unknown test name is an error, not a silent default.

TEST TERM | Directed test

A hand-written sequence of stimulus with a known expected result, aimed at one feature. Easy to debug, but it only checks what its author thought of. `directed` below is a directed test.

TEST TERM | Corner-case test

A directed test at the edges of the specification — here the operand values where multipliers break: 0, 1, all ones, the most negative value, powers of two, and requests back to back. Most datapath bugs live here; the corner cases are part of `directed`.

TEST TERM | Exhaustive test

Every possible input combination is applied — possible only for a small configuration: with `MUL_XLEN=8` there are 2^{16} operand pairs per operation ($4 \cdot 2^{16} = 262{,}144$ requests with the stretch operations, a few seconds of simulation). A pass is a proof of correctness for that configuration; the same RTL at XLEN = 32 is then trusted because nothing in it depends on the width except parameters.

`TEST`	Content
`smoke` (default)	Reset, then a few normal requests (MUL and MULHU) checked against `ref_mul`: is the block alive and does the flow work?
`directed`	Corner cases for every supported operation: $0\times$ anything, $1 \times x$, all-ones $\times$ all-ones, the most negative value (`'h8000_0000` for XLEN = 32) in each operand, powers of two, and two back-to-back requests (a new request right after a result).
`exhaustive`	Only with `DEFINES=MUL_XLEN=8`: every pair of operands for every supported operation (a nested loop over `a` and `b`). The test fails if `XLEN != 8`.
`comb`	A few directed vectors on `comb_mul`, checked against the full product two cycles after the operands are applied.

The simulation must end with exactly one pass/fail line and must have a watchdog timeout so a hang is reported as a failure.

7.4 Skeleton

The skeleton shows the structure only. Complete the TODOs, add the tests, and adapt it to your design. Note that the testbench may use `===` / `!==` (rule 7 applies to RTL only): a result containing X must be reported as a failure.

```
// hw/ip/seq_mul/tb/tb_seq_mul.sv
module tb_seq_mul;
  import mul_pkg::*;

  localparam int unsigned TimeoutCycles = 5_000_000;

  logic             clk = 1'b0;
  logic             rst_n;
  logic             in_valid, in_ready, out_valid, out_ready;
  mul_req_t         in_req;
  logic [XLEN-1:0] out_result;

  string       test_name = "smoke";
  int unsigned n_checks  = 0, n_errors = 0;
```

```
initial forever #5 clk = ~clk;

seq_mul u_dut (.*);
// TODO: instantiate comb_mul for the comb test

// ---------------- reference model ----------------
function automatic logic [XLEN-1:0] ref_mul(mul_op_e op,
                                            logic [XLEN-1:0] a, b);
  logic [2*XLEN-1:0] p;
  unique case (op)
    OP_MUL, OP_MULHU:
      p = {{XLEN{1'b0}}, a} * {{XLEN{1'b0}}, b};
    OP_MULH:
      p = $signed({{XLEN{a[XLEN-1]}}, a}) * $signed({{XLEN{b[XLEN-1]}}, b});
    OP_MULHSU:
      p = $signed({{XLEN{a[XLEN-1]}}, a}) * $signed({{XLEN{1'b0}}, b});
    default: p = 'x;
  endcase
  return (op == OP_MUL) ? p[XLEN-1:0] : p[2*XLEN-1:XLEN];
endfunction

// ---------------- reset ----------------
task automatic apply_reset();
  rst_n     = 1'b0;
  in_valid  = 1'b0;
  in_req    = '0;
  out_ready = 1'b1;
  repeat (3) @(posedge clk);
  @(negedge clk);
  rst_n = 1'b1;
endtask

// ---------------- one request: drive, wait, check ----------------
task automatic do_mul(mul_op_e op, logic [XLEN-1:0] a, b);
  logic [XLEN-1:0] exp;
  exp = ref_mul(op, a, b);
  @(negedge clk);                          // drive away from the active edge
  in_req   = '{op: op, a: a, b: b};
  in_valid = 1'b1;
  do @(posedge clk); while (!in_ready);    // accepted on this edge
  @(negedge clk);
  in_valid = 1'b0;
  do @(posedge clk); while (!out_valid);   // out_ready = 1: taken on this edge
  n_checks++;
  if (out_result !== exp) begin
    n_errors++;
    $error("%s a=%h b=%h: result %h, expected %h",
           op.name(), a, b, out_result, exp);
  end
endtask

// ---------------- report ----------------
function void report();
  $display("CHECKS: %0d  ERRORS: %0d", n_checks, n_errors);
  if (n_checks > 0 && n_errors == 0) $display("TEST PASSED");
  else                               $display("TEST FAILED");
endfunction

// ---------------- watchdog ----------------
```

```
  initial begin
    repeat (TimeoutCycles) @(posedge clk);
    $display("ERROR: watchdog timeout after %0d cycles", TimeoutCycles);
    $display("TEST FAILED");
    $finish;
  end

  // ---------------- tests ----------------
  task automatic run_directed();
    // TODO: corner cases for every supported operation
    do_mul(OP_MUL,   '0, '1);
    do_mul(OP_MULHU, '1, '1);
  endtask

  task automatic run_exhaustive();
    if (XLEN != 8) begin
      n_errors++;
      $error("exhaustive test requires DEFINES=MUL_XLEN=8");
      return;
    end
    for (int a = 0; a < 256; a++)
      for (int b = 0; b < 256; b++) begin
        do_mul(OP_MUL,   XLEN'(a), XLEN'(b));
        do_mul(OP_MULHU, XLEN'(a), XLEN'(b));
      end
  endtask

  initial begin
    void'($value$plusargs("TEST=%s", test_name));
    $display("CONFIG: TEST=%s XLEN=%0d", test_name, XLEN);
    apply_reset();
    case (test_name)
      "smoke":      run_smoke();          // TODO: a few normal requests
      "directed":   run_directed();
      "exhaustive": run_exhaustive();
      // TODO: "comb"
      default: begin
        n_errors++;
        $error("unknown TEST=%s", test_name);
      end
    endcase
    report();
    $finish;
  end
endmodule
```

NOTE | Why `do_mul` reads outputs right after `@(posedge clk)`.

A process that wakes up on the rising edge reads every signal before the flip-flops of that edge have updated. `in_ready` and `out_valid` therefore show the values the DUT used to decide the handshake at this edge — the same values a flip-flop in the consumer would capture. Lab R2 explains this in terms of scheduling regions.

8 Requirements and Tasks

1. Create the branch `lab/r1-seq-mul` from your main branch.
2. Write `doc/spec.md` (operations, interface, behavior) before writing RTL.

3. Write `doc/design.md`: the register mapping table of Section 2.2 for your design, a datapath block diagram, the FSM state diagram, and the latency in cycles.
4. Implement `rtl/mul_pkg.sv` and `rtl/seq_mul.sv` with MUL and MULHU. MULH and MULHSU are stretch goals; if you do not implement them, the unit must still accept the encodings and return a defined value (document which).
5. Use every construct of the required-constructs table and follow the lowRISC style guide and Rules 1–12.
6. Implement `rtl/comb_mul.sv`.
7. Make `make lint` clean for `DUT_TOP=seq_mul` and `DUT_TOP=comb_mul`, at XLEN 32 and 8. Every waiver in `lint/seq_mul.vlt` has a justification comment.
8. Do the lint exercise of Section 6 and record the results.
9. Write `tb/tb_seq_mul.sv` and `tb/tb.f`. The tests `smoke`, `directed`, `comb` and `exhaustive` (with `MUL_XLEN=8`) must pass, and `make sim` must return a non-zero exit code when a check fails (prove it once by breaking `ref_mul` temporarily).
10. Look at the waveform of one MULHU request (`WAVES=1`) and check that the shift register, counter and FSM behave as in `doc/design.md`.
11. Write the README, open the merge request, and tag `r1-done` after approval. `make -C $REPO_ROOT check-all` must pass.

Synthesis, gate-level simulation and timing of `seq_mul` and `comb_mul` are done in Labs P1 and P2; nothing in this lab requires Yosys or OpenSTA.

9 Repository and Flow

9.1 Files created in this lab

```
hw/ip/seq_mul/
+-- README.md              block summary, how to run every target, results, limitations
+-- Makefile               IP variables + include of flow/mk/*.mk
+-- doc/
|   +-- spec.md            operations, interface, behavior
|   +-- design.md          register mapping, block and FSM diagrams, latency, lint demo
+-- rtl/
|   +-- mul_pkg.sv
|   +-- seq_mul.sv
|   +-- comb_mul.sv
|   +-- seq_mul.f          RTL file list: synthesizable files only
+-- lint/
|   +-- seq_mul.vlt        Verilator waivers, each justified
|   +-- lint_demo_bad.sv   lint exercise, not in any file list
+-- tb/
    +-- tb_seq_mul.sv      directed smoke testbench
    +-- tb.f
```

No file outside `hw/ip/seq_mul/` changes, except the list of IPs checked by `check-all` and the lab status in the top-level `README.md`.

9.2 File lists

```
// hw/ip/seq_mul/rtl/seq_mul.f
+incdir+${REPO_ROOT}/hw/ip/seq_mul/rtl
${REPO_ROOT}/hw/ip/seq_mul/rtl/mul_pkg.sv
${REPO_ROOT}/hw/ip/seq_mul/rtl/seq_mul.sv
${REPO_ROOT}/hw/ip/seq_mul/rtl/comb_mul.sv
```

```
// hw/ip/seq_mul/tb/tb.f
-f ${REPO_ROOT}/hw/ip/seq_mul/rtl/seq_mul.f
${REPO_ROOT}/hw/ip/seq_mul/tb/tb_seq_mul.sv
```

The package comes before the modules that import it. `MUL_XLEN` is not set in the file list; it comes from `DEFINES`, so the same list serves every width.

9.3 IP Makefile

```
# hw/ip/seq_mul/Makefile
IP         := seq_mul
# comb_mul: make lint DUT_TOP=comb_mul
DUT_TOP    ?= seq_mul
TB_TOP     ?= tb_seq_mul
RTL_FLIST  := rtl/seq_mul.f
# V phase: dv/dv.f, TB_TOP=tb_top
TB_FLIST   ?= tb/tb.f
TEST       ?= smoke
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
# include $(REPO_ROOT)/flow/mk/regress.mk    (V1)
# include $(REPO_ROOT)/flow/mk/syn.mk        (P1)
# include $(REPO_ROOT)/flow/mk/sta.mk        (P2)
```

`DUT_TOP` is overridable (`?=`) because this IP contains two top-level designs; this is the only difference from the template of Lab L0. Comments go on their own lines: in `make`, the blanks before a trailing `#` comment become part of the value.

9.4 Commands

No new targets or variables are introduced in this lab; you use the L0 targets `lint`, `build`, `sim`, `waves`, `clean` and the variables `DEFINES`, `TEST` and `WAVES`.

```
source env.sh                           # from the repository root
cd $REPO_ROOT/hw/ip/seq_mul

make lint                               # seq_mul, XLEN = 32
make lint DUT_TOP=comb_mul
make lint DEFINES=MUL_XLEN=8

make sim                                # TEST=smoke
make sim TEST=directed
make sim TEST=comb
make sim DEFINES=MUL_XLEN=8 TEST=exhaustive
make sim TEST=directed WAVES=1 && make waves

make -C $REPO_ROOT lint sim IP=seq_mul   # same, from the top level
make -C $REPO_ROOT check-all             # every IP still passes
```

SYNOPSYS CLOUD

`make sim SIM=vcs` builds and runs the same testbench with VCS (the cloud default; `SIM=verilator` selects Verilator). VCS is four-state: an uninitialized datapath register shows up as X in `out_result`, and the testbench's `!==` reports it. `make sim WAVES=1` then `make waves` opens the FSDB in Verdi, which also shows the FSM `state_q` by name and the source code next to the waveform. Measured with the reference design: the exhaustive test (262,145 checks) runs in about 7 s on VCS and 16 s on Verilator 5.013.

Outputs go to `build/seq_mul/<cfg>/{lint,sim}/`, where `<cfg>` is `default` or, for example, `MUL_XLEN-8`. Because the configuration tag contains the defines, the 8-bit and 32-bit models are compiled into different directories and never mixed up.

10 Deliverables

The merge request from `lab/r1-seq-mul` must contain:

- All files of the tree above; nothing generated (`build/` is never committed).
- `doc/spec.md`: operations, interface table, behavior, supported operations and the defined behavior of unsupported ones.
- `doc/design.md`: register mapping table, datapath block diagram, FSM state diagram, latency, and the lint demo results (warning text, rule, consequence, fix) for rules 1, 3, 4, 7 and 12.
- `README.md`: what the block is, how to run every `make` target used in this lab, test results (test name, number of checks, pass/fail, for XLEN 32 and 8), any deviation from the lowRISC style guide with its reason, and known limitations (for example, unimplemented stretch operations).
- A clean lint log and passing simulation logs, reproducible from a clean checkout with the commands above.

11 Assessment

Criterion	Weight
Functional correctness: smoke, directed, comb and exhaustive tests pass	30%
RTL quality: required constructs, Rules 1–12, lowRISC style, lint-clean	25%
Micro-architecture and design documentation (spec.md, design.md)	15%
Smoke testbench: structure, corner cases, self-checking, watchdog	15%
Lint exercise: violations shown, mapped to rules and explained	10%
Reproducibility and Git workflow	5%
Total	100%

12 Review Questions

The coordinator will ask questions like these in the review. Be ready to answer them without notes.

1. Point to every flip-flop in your RTL. Why does each one need to exist, and which of them are reset?
2. Which of your `always_comb` blocks would infer a latch if you removed the default assignments?

3. Why did you choose `unique case` here and not `priority case`? What does the simulator do when a `unique case` has no matching item?
4. What does synthesis do with an enum state that has an unused encoding, and how does your `default` item protect the FSM?
5. Why is `XLEN` a package `localparam` set from a macro and not a module parameter?
6. Rewrite your multiplier datapath as a `for` loop in `always_comb`. What hardware would that describe, and how many cycles would it take?
7. Why does the testbench rely on the handshake and not on the latency of your design?
8. Where could your testbench and DUT race, and what prevents it?
9. Which of the five violations in your lint demo would also be a simulation–synthesis mismatch, and which is "only" bad hardware?
10. Your smoke tests pass. Which bugs could still be hiding, and which feature of Lab V1 would find them?

13 References

Coding style:

- lowRISC, Verilog/SystemVerilog Coding Style Guide.

Simulation–synthesis mismatches (the sources below are partly written for Verilog; this lab applies their ideas to SystemVerilog):

- Clifford E. Cummings, RTL Coding Styles That Yield Simulation and Synthesis Mismatches, SNUG 1999.
- Circuit Quest, The Sim/Synth Mismatch Checklist.
- VLSI-Design, RTL coding styles that lead to pre- and post-synthesis simulation mismatch.

Language and architecture:

- *SystemVerilog for Design*, Sutherland, Davidmann and Flake: chapters on procedural blocks, enumerated types and structures.
- RISC-V unprivileged specification, M extension: definitions of MUL, MULH, MULHSU and MULHU.
- GeeksforGeeks, Sequential Binary Multiplier.

Tools:

- Verilator User's Guide: `--lint-only`, the list of warnings, and configuration files (`.vlt`) for waivers.